



汇编语言与微机接口技术

第4章 汇编语言程序设计

2025年2月10日



信息与软件工程学院
School of Information and Software Engineering

4.10

● 源程序的两种结构

4.10 源程序的两种结构（执行DOS功能调用4CH返回操作系统）

数据段	{	DATA SEGMENT	
			
		DATA ENDS	;定义数据段
堆栈段	{	STACK1 SEGMENT PARA STACK	
		DW 20H DUP (0)	
		STACK1 ENDS	;定义堆栈段
代码段	{	CODE SEGMENT	
		ASSUME CS:CODE,DS:DATA,SS:STACK1	
		START:	;指令开始地址
		MOV AX,DATA	
		MOV DS,AX	;初始化DS
			
		MOV AH, 4CH	
		INT 21H	;返回DOS操作系统
		CODE ENDS	
		END START	;汇编结束标志

4.10 源程序的两种结构（使用程序段前缀PSP返回操作系统）



数据段	{	<pre>DATA SEGMENT DATA ENDS ;定义数据段</pre>
堆栈段	{	<pre>STACK1 SEGMENT PARA STACK DW 20H DUP (0) STACK1 ENDS ;定义堆栈段</pre>
代码段	{	<pre>CODE SEGMENT ASSUME CS:CODE,DS:DATA,SS:STACK1 MAIN PROC FAR ;设置为FAR过程 PUSH DS ;为返回操作系统执行INT 20H MOV AX,0 ;指令做准备 PUSH AX ;立即数不能够作为操作数 RET ;返回操作系统 MAIN ENDP CODE ENDS END MAIN ;汇编结束标志</pre>



4.11

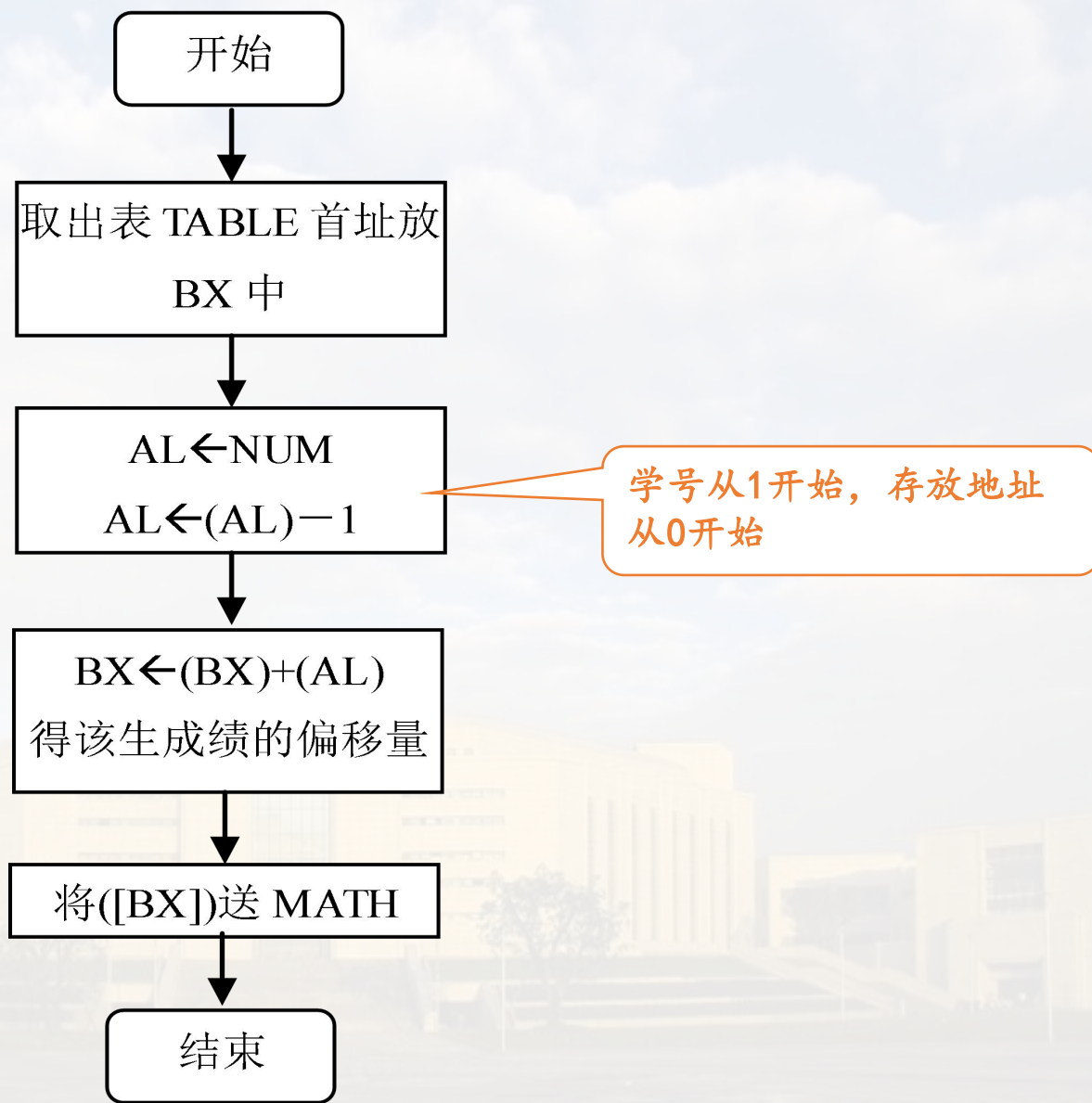
顺序程序设计实例

例1 利用学号查学生的数学成绩表。

算法分析：首先在数据段中建立一个成绩表TABLE，在表中各学生的成绩按照学号从小到大的顺序存放。要查的学号存放在变量NUM中，查表的结果放在变量MATH中。

A large, stylized blue pen is positioned diagonally across the bottom right of the slide, pointing towards the bottom right corner. The pen has a blue body with white rings and a white eraser at the top.

4.11 顺序程序设计实例



4.11 顺序程序设计实例

```

    TITLE    TABLE      LOOKUP
DATA SEGMENT
TABLE DB 81, 78, 90, 64, 85, 76, 93, 82, 57, 80
      DB 73, 62, 87, 77, 74, 86, 95, 91, 82, 71
NUM   DB 8
MATH  DB ?
DATA  ENDS

STACK1 SEGMENT PARA STACK
      DW 20H DUP (0)
STACK1 ENDS
```

查学号是8号的同学成绩

20位同学的成绩

4.11 顺序程序设计实例

```
COSEG SEGMENT
    ASSUME CS:COSEG, DS:DATA, SS:STACK1
START: MOV AX, DATA    ;
        MOV DS, AX      ;装入DS
        MOV BX, OFFSET TABLE    ;BX指向表首址
        XOR AH, AH        ;(AH)=0
        MOV AL, NUM
        DEC AL            ;实际学号是从1开始的
        ADD BX, AX        ;BX加上学号指向要查的成绩
        MOV AL, [BX]      ;查到成绩送AL
        MOV MATH, AL      ;存结果
        MOV AH, 4CH       ;返回DOS
        INT 21H
COSEG ENDS
    END START
```

可用指令
XLAT替代?

4.12

分支程序实例

» 常见结构

两种常
见结构

用比较/测试指令+条件转
移指令实现分支

用跳转表形成多路分支

4.12 分支程序实例

» 用比较/测试指令+条件转移指令实现分支

 比较指令: **CMP DEST, SRC**

该指令的功能与减法指令SUB相似，区别是 $(DEST) - (SRC)$ 的差值不送入DEST。而其结果影响标志位。

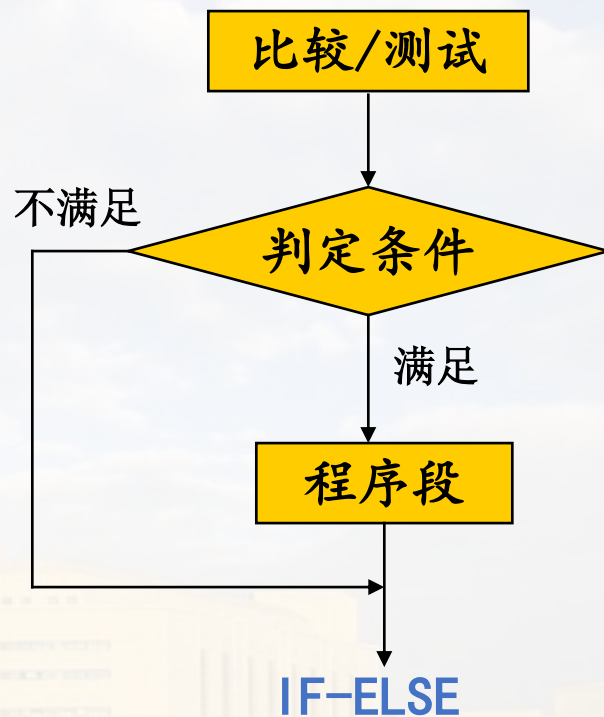
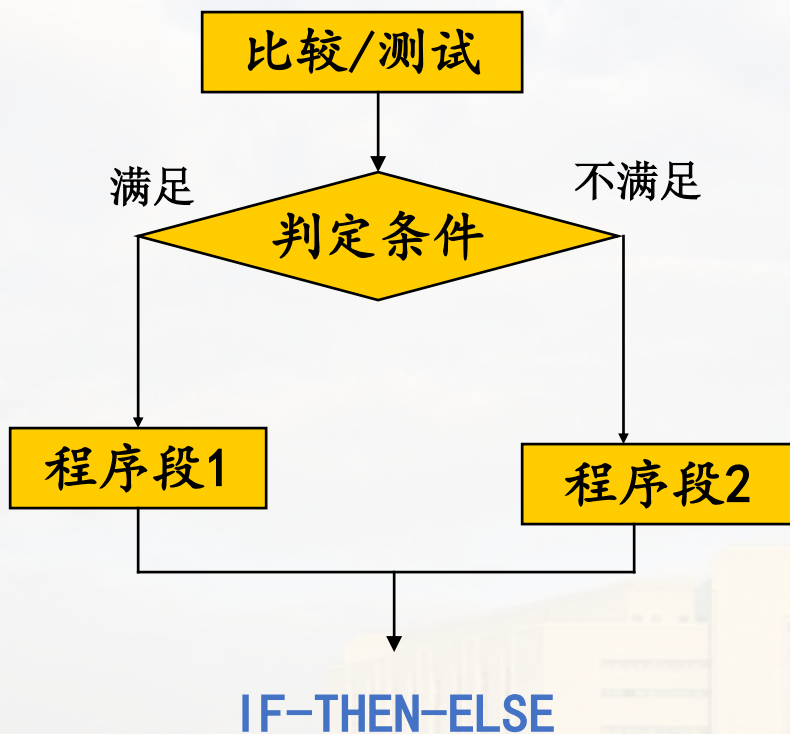
 测试指令: **TEST DEST, SRC**

该指令的功能与逻辑指令AND相似，区别是逻辑“与”的结果不送入DEST。只是结果影响标志位。

4.12 分支程序实例

» 用比较/测试指令+条件转移指令实现分支

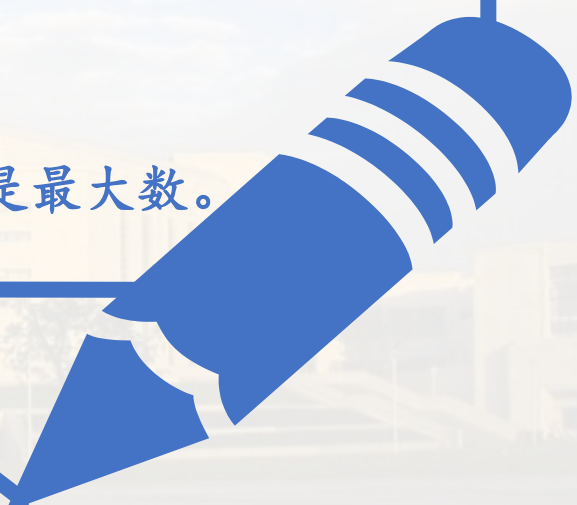
这种类型的分支程序有两种结构



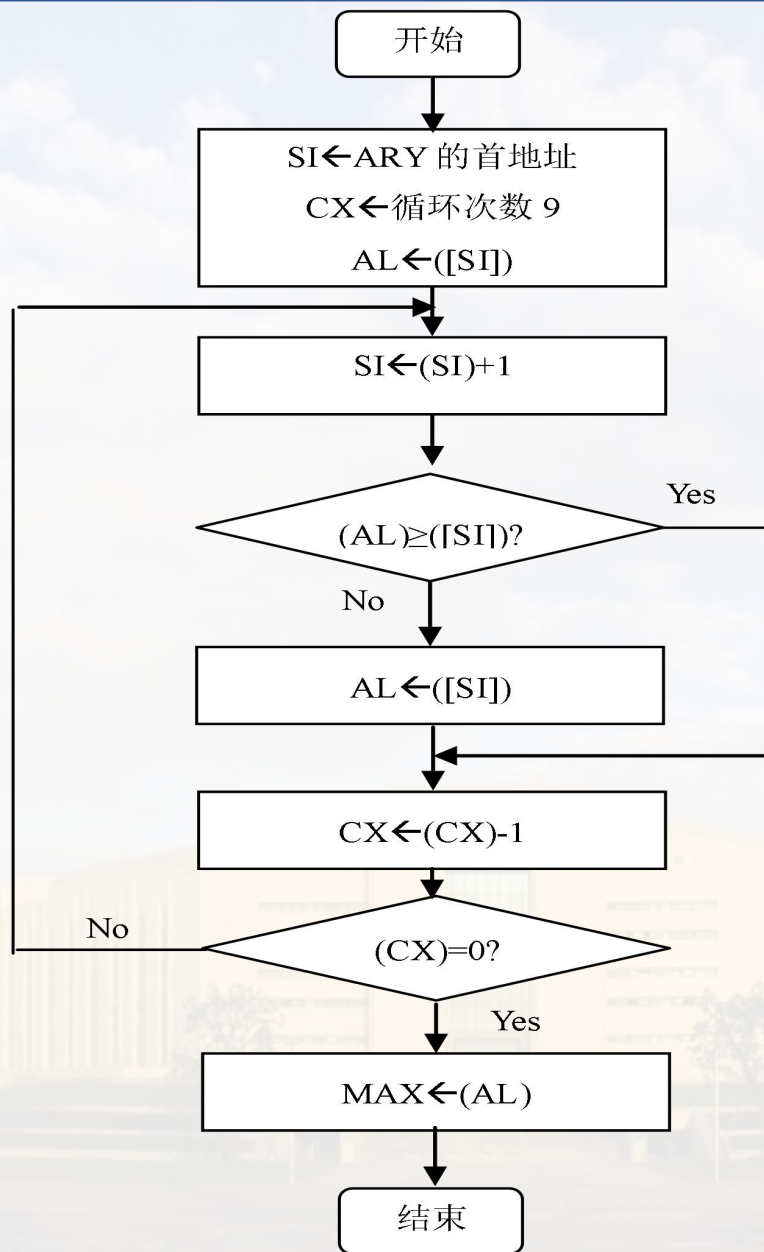
一条条件转移指令只能实现两条分支的程序。要实现更多条分支的程序，需使用多条条件转移指令。

例2 数据段的ARY数组中存放有10个无符号数，试找出其中最大者送入MAX单元。

算法分析：

- 依次比较相邻两数的大小，将较大的送入AL中。
 - 每次比较后，较大数存放在AL中，相当于较大的数往下传。
 - 比较一共要做9次。
 - 比较结束后，AL中存放的就是最大数。
- 

4.12 分支程序实例



4.12 分支程序实例

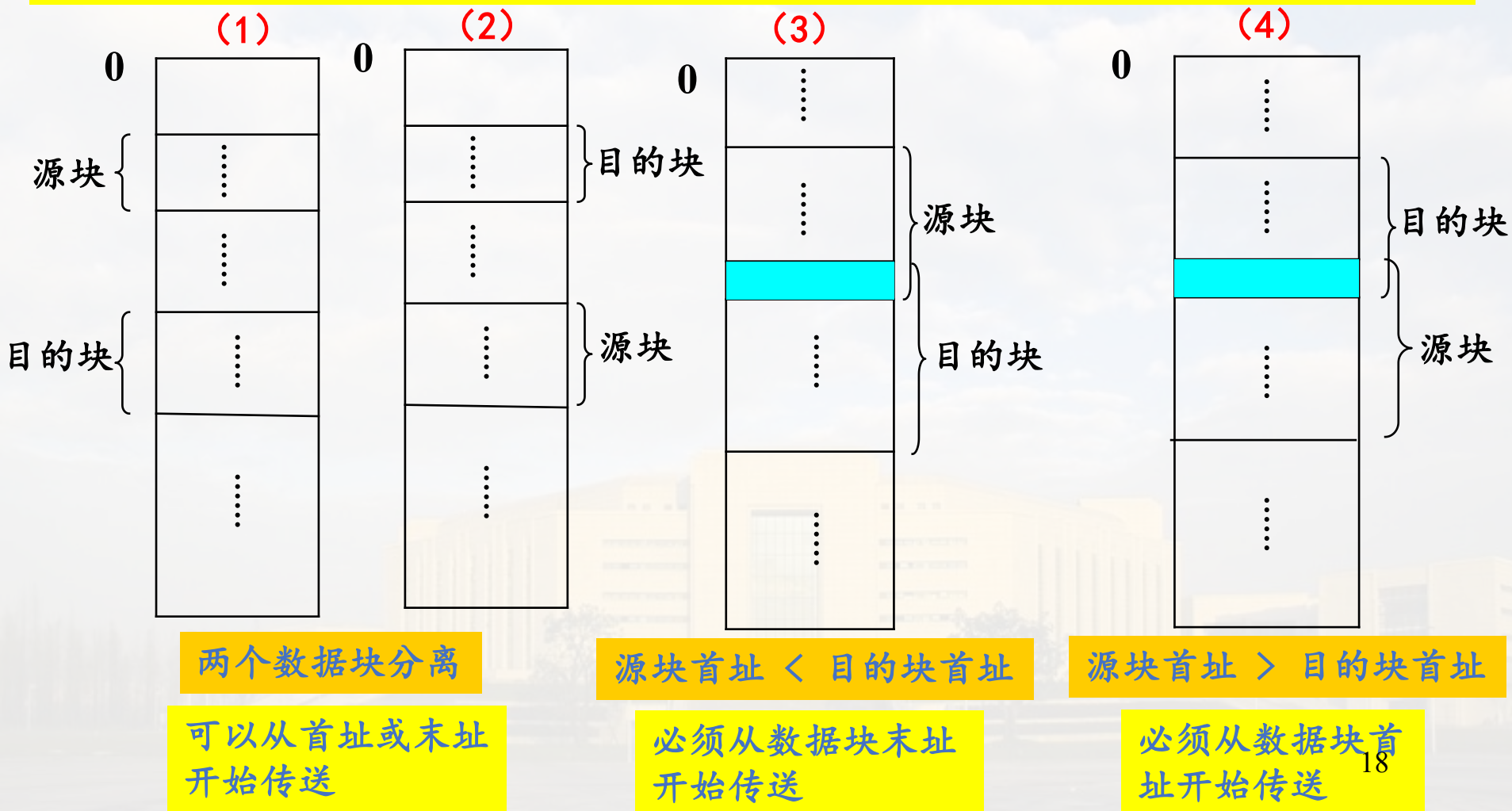


```
DATA SEGMENT
    ARY DB 17, 5, 40, 0, 67, 12, 34, 78, 32, 10
    MAX DB ?
DATA END
STACK1 SEGMENT PARA STACK
    DW 20H DUP (0)
STACK1 ENDS
CODE SEGMENT
    ASSUME CS:CODE, DS:DATA, SS:STACK1
BEGIN:
    MOV AX, DATA
    MOV DS, AX
    MOV SI, OFFSET ARY ; SI指向ARY的第一个元素
    MOV CX, 9 ; CX作次数计数器
    MOV AL, [SI] ; 取第一个元素到AL
LOP: INC SI ; SI指向后一个元素
    CMP AL, [SI] ; 比较两个数
    JAE BIGER ; 前元素 ≥ 后元素转移
    MOV AL, [SI] ; 取较大数到AL
BIGER: DEC CX ; 减1计数
    JNZ LOP ; 未比较完转回去, 否则顺序执行
    MOV MAX, AL ; 存最大数
    MOV AH, 4CH
    INT 21H
CODE ENDS
END BEGIN
```

4.12 分支程序实例：比较PPT3.5串操作指令的例题P56

例3 编写一程序，实现将存储器中的源数据块传送到目的数据块。

在存储器中两个数据块的存放有下列情况：两个数据块分离和有部分重叠。



三种相对位置情况的传送方法归纳

归纳

01

对于源块和目的块分离的情况，不论是从数据块的首址还是末址开始传送都可以。

02

对于源块与目的块有重叠且源块首址 $<$ 目的块首址的情况，必须从数据块末址开始传送。

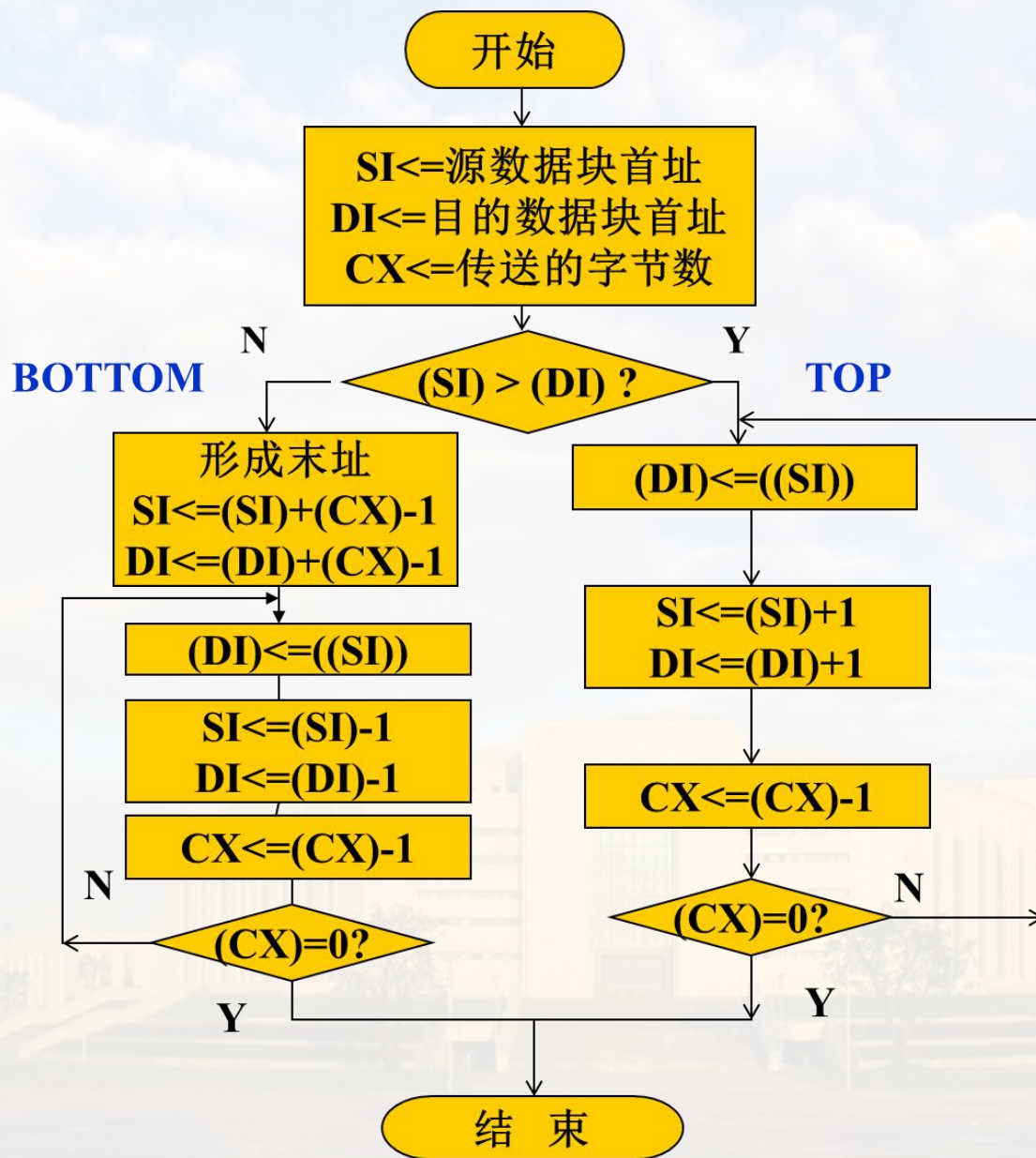
03

对于源块与目的块有重叠且源块首址 $>$ 目的块首址的情况，必须从数据块首址开始传送。

(1) 是 (3) 的特例，
(2) 是 (4) 的特例

因此，我们设定：当源块首地址 $<$ 目的块首地址时，从数据块末地址开始传送。反之，则从首地址开始传送。

4.12 分支程序实例



4.12 分支程序实例



```
TITLE DATA BLOCK MOVE
DATA SEGMENT
ORG $+20H
STRG DB 'ABCDEFGH I J' ; 数据块
LENG EQU $-STRG ; 数据块字节长度
BLOCK1 DW STRG ; 源块首址
BLOCK2 DW STRG-5 ; 目的块首址
```

```
DATA ENDS
```

本题假设，BLOCK1地址高，BLOCK2地址低，且相差5B

```
STACK1 SEGMENT STACK
```

```
DW 20H DUP (0)
```

```
STACK1 ENDS
```

注意：变量在伪指令语句中的用法，能否用DB？

```
COSEG SEGMENT
```

```
    ASSUME CS:COSEG, DS:DATA, SS:STACK1
```

```
BEGIN: MOV AX, DATA
```

```
        MOV DS, AX
```

```
        MOV CX, LENG           ;设置计数器初值
```

```
        MOV SI, BLOCK1         ;SI指向源块首址
```

```
        MOV DI, BLOCK2         ;DI指向目的块首址
```

```
        CMP SI, DI             ;源块首址>目的块首址吗?
```

```
        JA  TOP                 ;大于则转到TOP处, 否则顺序执行
```

```
        ADD SI, LENG-1          ;SI指向源块末址
```

```
        ADD DI, LENG-1          ;DI指向目的块末址
```

4.12 分支程序实例



BOTTOM: MOV AL, [SI] ;从末址开始传送

MOV [DI], AL

DEC SI

DEC DI

DEC CX

JNE BOTTOM

JMP END1

TOP: MOV AL, [SI] ;从首址开始传送

MOV [DI], AL

INC SI

INC DI

DEC CX

JNE TOP

END1: MOV AH, 4CH

INT 21H

COSEG ENDS

END BEGIN

Rep MOVSB ?

» 用跳转表形成多路分支

当程序的分支数量较多时，采用跳转表的方法可以使程序长度变短，跳转表有两种构成方法：

跳转表用**入口地址**构成

存放在DS段中

跳转表用**无条件转移指令**构成

存放在CS段中

» 跳转表用入口地址构成

方法：将各分支的**入口地址**组织成一个表放在**数据段**中，在程序中通过查表的方法获得各分支的入口地址。

4.12 分支程序实例

例4 设某程序有10路分支，试根据变量N的值（1~10），控制程序转移到其中的一路分支执行。

设10路分支程序段的入口地址分别为：BRAN1、BRAN2.....BRAN10。

当变量N为1时，转移到BRAN1；N为2时，转移到BRAN2，依次类推。

在跳转表中每两个字节存放一个入口地址的偏移量。

程序中，先根据N的值形成查表地址：表首址+ $(N-1) \times 2$ 。

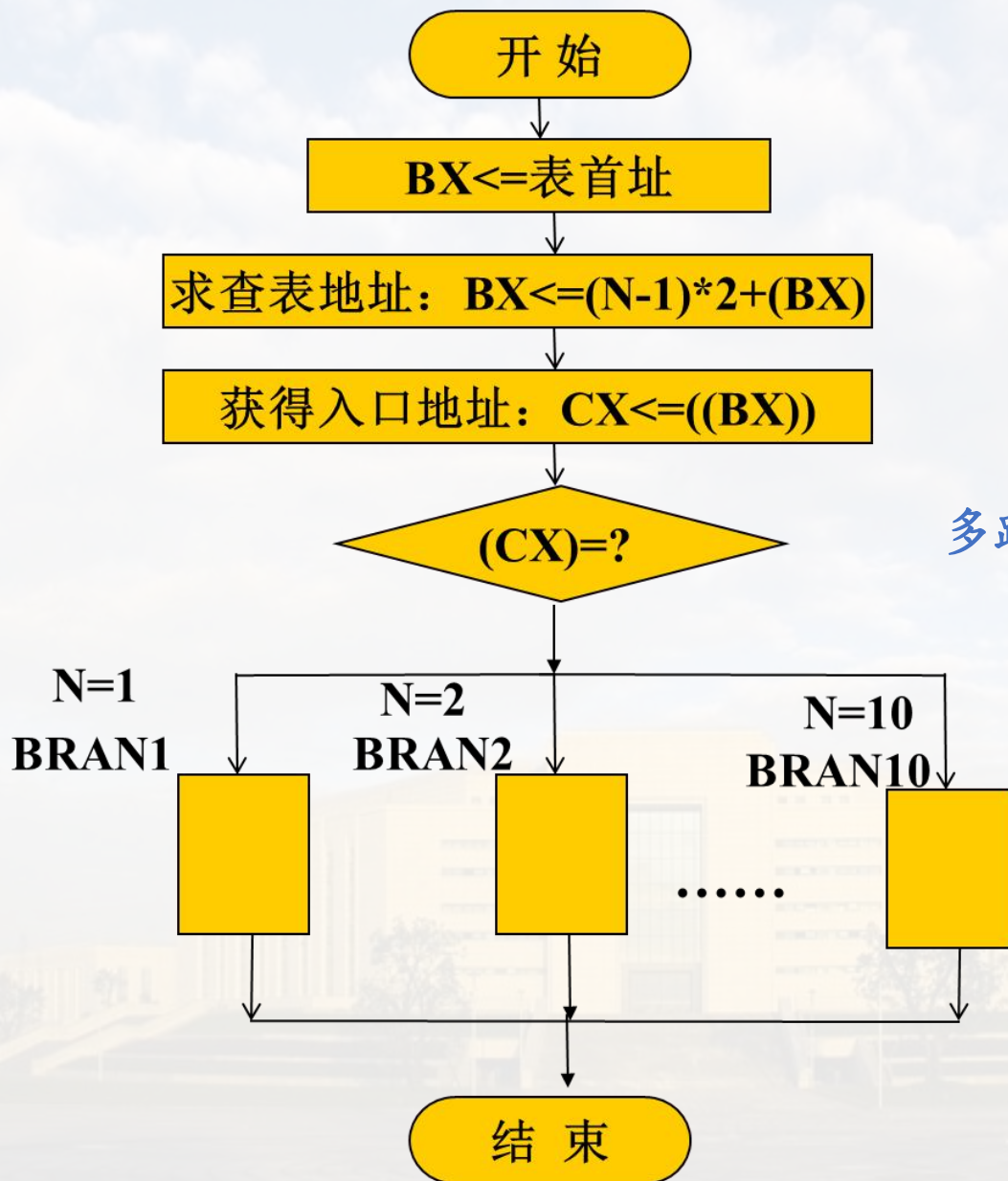
数据段	
表首址→	BRAN1_
	偏移量
	BRAN2_
	偏移量
	BRAN3_
	偏移量
	⋮
	BRAN10_
	偏移量

注意是DS，且该题是段内转移

与中断向量表中的中断向量类似

跳转表

4.12 分支程序实例



多路分支结构流程图

4.12 分支程序实例



TITLE JUMP TABLE OF ADDRESS

DATA SEGMENT

ATABLE DW BRAN1, BRAN2, BRAN3, ..., BRAN10

N DB 3 ; 假如为3号分支

DATA ENDS

STACK1 SEGMENT PARA STACK

DW 20H DUP (0)

STACK1 ENDS

CODE SEGMENT

ASSUME CS:CODE, DS:DATA, SS:STACK1

START: MOV AX, DATA

MOV DS, AX

注意：变量在伪指令语句中的用法，能否用DB？

4.12 分支程序实例

```
XOR  AH, AH
MOV  AL, N
DEC  AL
SHL  AL, 1                ;  $(N-1) \times 2$ 
MOV  BX, OFFSET ATABLE   ; BX指向表首址
ADD  BX, AX               ; BX指向查表地址
MOV  CX, [BX]             ; 将N对应的分支入口地址送到CX中
JMP  CX                   ; 转移到N对应的分支入口地址
```

XLAT代替?

分支地址是16位

4.12 分支程序实例



BRAN1:

.....

JMP END1

BRAN2:

.....

JMP END1

BRAN3:

.....

JMP END1

⋮

BRAN10:

.....

END1: MOV AH, 4CH

INT 21H

CODE ENDS

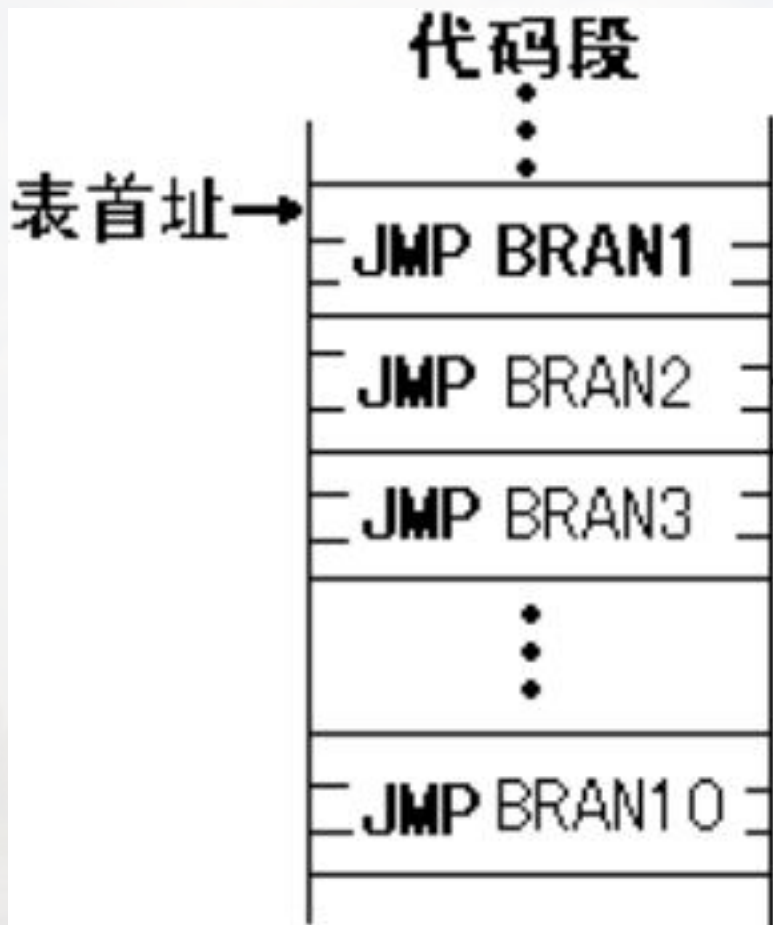
END START

分支结束时，要转到
DOS调用，否则，要顺
序执行后面的指令

4.12 分支程序实例

» 跳转表用无条件转移指令构成

跳转表中的每一项是一条无条件转移指令。跳转表是代码段中的一段程序。



注意是CS，且该题是段内转移

每条JMP指令为3个字节。

查表地址：表首址 + (N-1) × 3

4.12 分支程序实例



例 4的源程序可修改为如下程序：

```
TITLE  JUMP TABLE OF INSTRUCTION
```

```
DATA  SEGMENT
```

```
    ATABLE  DW  BRAN1, BRAN2, BRAN3, ..., BRAN10  (3字节? ? ? )
```

```
    N  DB  3                      ; 假如为3号分支
```

```
DATA  ENDS
```

```
STACK1 SEGMENT PARA STACK
```

```
    DW  20H DUP (0)
```

```
STACK1  ENDS
```

```
CODE  SEGMENT
```

```
    ASSUME  CS:CODE, DS:DATA, SS:STACK1
```

```
START:MOV  AX, DATA
```

```
        MOV  DS, AX
```

```
        MOV  BH, 0
```

```
        MOV  BL, N
```



```
    { DEC BL                      ;四条指令实现 (N-1)*3
      MOV AL, BL
      SHL BL, 1
      ADD BL, AL
```

```
    ADD BX, OFFSET ITABLE ;BX指向查表地址
```

```
    JMP BX                ;转移到N对应的JMP指令
```

```
ITABLE: JMP BRAN1          ;JMP指令构成的跳转表
        JMP BRAN2          ;每一条指令都是3字节的编码
        JMP BRAN3
        :
        :
        JMP BRAN10
```

4.12 分支程序实例



BRAN1 :

.....

JMP END1

BRAN2:

.....

JMP END1

BRAN3:

.....

JMP END1

⋮

BRAN10:

.....

END1: MOV AH, 4CH

INT 21H

CODE ENDS

END START

» 用跳转表形成多路分支总结

跳转表用入口地址构成：存放在DS中

入口地址段内占2B，因此，分支地址为 $(N-1) \times 2$

入口地址段间占4B，因此，分支地址为 $(N-1) \times 4$

跳转表用无条件转移指令构成：存放在CS中

入口地址段内占3B，因此，分支地址为 $(N-1) \times 3$

入口地址段间占5B，因此，分支地址为 $(N-1) \times 5$

4.13

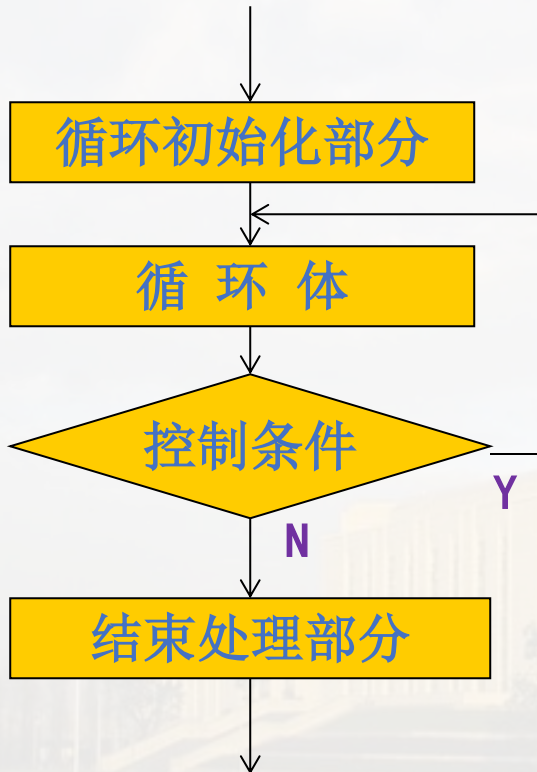
● 循环程序实例

4.13 循环程序实例

循环程序两种结构形式

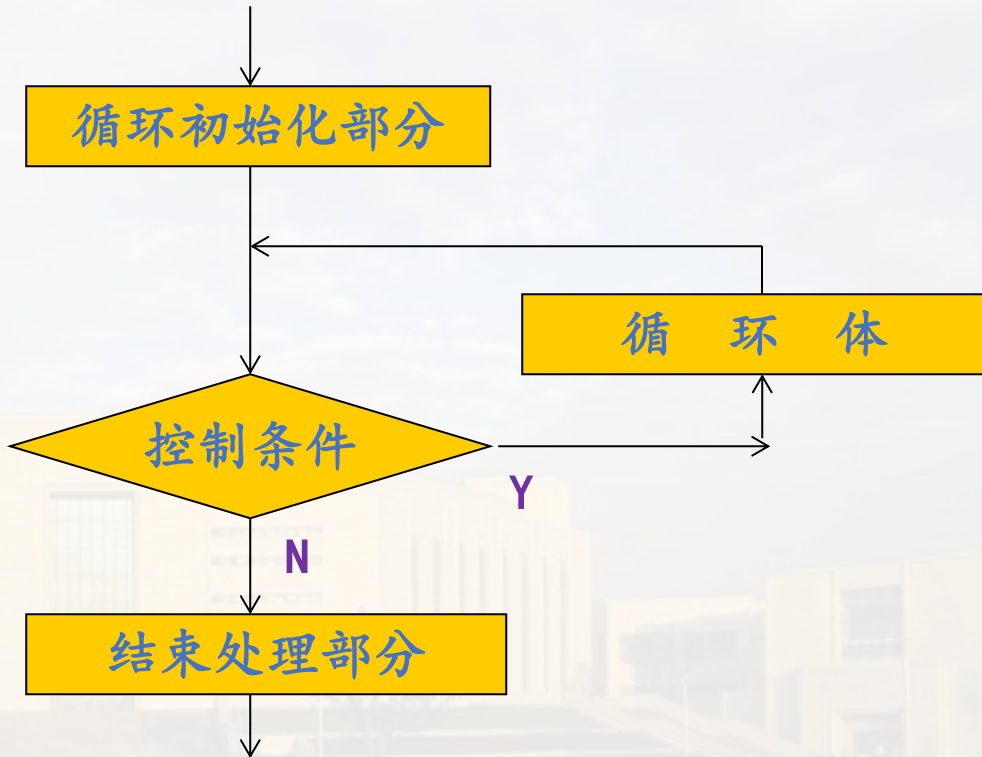
至少执行一次循环

1、先执行后判断结构



可能一次循环也不执行

2、先判断后执行结构



» 在循环程序中主要包括以下四个部分：



初始化部分



循环体



循环控制部分



结束处理部分

初始化部分：用于建立循环的初始状态。包括：循环次数计数器、（源/目的）地址指针以及其它循环参数的初始设定。

4.13 循环程序实例

» 在循环程序中主要包括以下四个部分



初始化部分



循环体



循环控制部分



结束处理部分

循环体：循环程序完成的主要任务。包括**工作部分**和**修改部分**。

工作部分：是完成循环程序任务的主要程序段。

修改部分：为循环的重复执行，完成某些参数的修改。

» 在循环程序中主要包括以下四个部分



初始化部分



循环体



循环控制部分



结束处理部分

循环控制部分：判断循环条件是否成立。可以有以下两种判断方法：

(1) 用计数控制循环——循环次数已知的情况：如LOOP等

(2) 用条件控制循环——循环次数未知的情况：JMP

» 在循环程序中主要包括以下四个部分



初始化部分



循环体



循环控制部分



结束处理部分

结束处理部分：处理循环结束后的结果。如存储结果等。

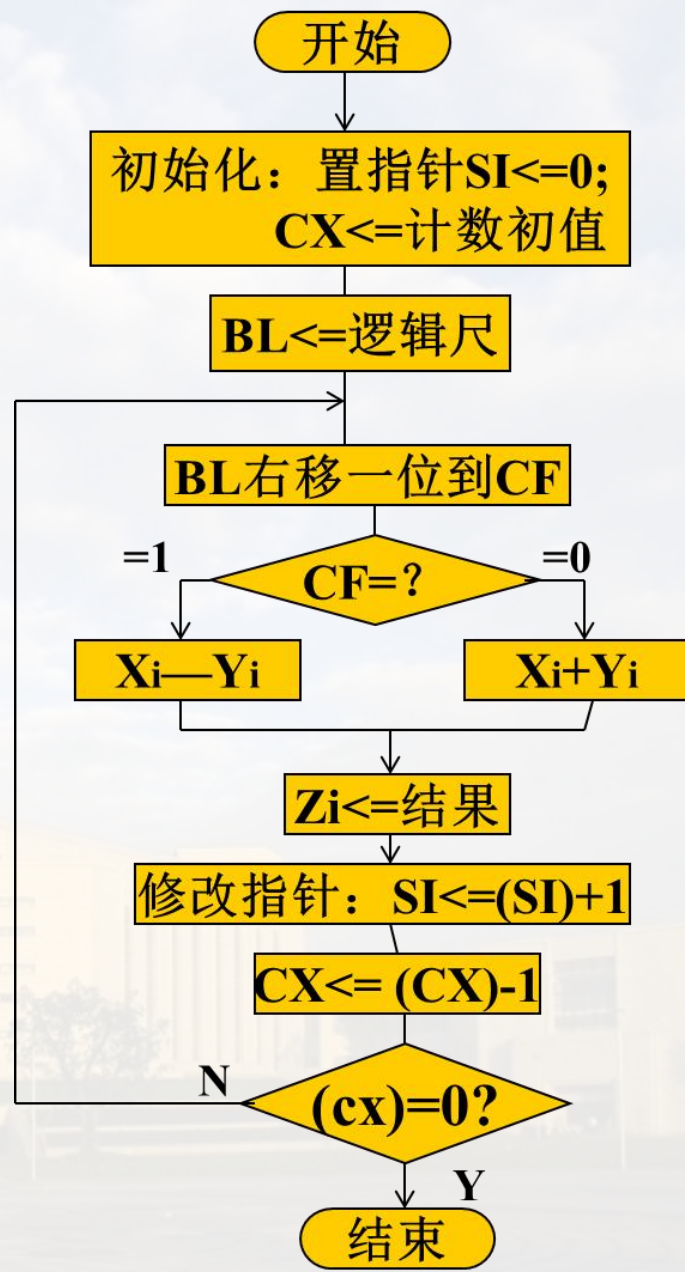
4.13 循环程序实例

例5 设有两个数组X和Y，它们都有8个元素，其元素按下标从小到大的顺序存放在数据段中。试编写程序完成下列计算：

$$\begin{array}{lll} Z1=X1+Y1 & Z2=X2-Y2 & Z3=X3+Y3 \\ Z4=X4-Y4 & Z5=X5-Y5 & Z6=X6+Y6 \\ Z7=X7+Y7 & Z8=X8-Y8 & \end{array}$$

由于循环体中“+”和“-”两种运算不是按简单规律出现，可通过设置标志0(+)和1(-)来确定。使用一个8位逻辑尺来对应八个运算表达式：10011010B

方向反向



4.13 循环程序实例

DATA SEGMENT

X DB 0A2H, 7CH, 34H, 9FH, 0F4H, 10H, 39H, 5BH

Y DB 14H, 05BH, 28H, 7AH, 0EH, 13H, 46H, 2CH

LEN EQU \$ -Y

Z DB LEN DUP(?)

LOGR DB 10011010B; 设置标志0(+)和1(-)来判断, 低位表示低下标的运算

DATA ENDS

STACK0 SEGMENT PARA STACK

DW 20H DUP(0)

STACK0 ENDS

COSEG SEGMENT

ASSUME CS:COSEG, DS:DATA, SS:STACK0

BEGIN: MOV AX, DATA

MOV DS, AX

MOV CX, LEN ; 初始化计数器

MOV SI, 0 ; 初始化指针

MOV BL, LOGR ; 初始化逻辑尺

4.13 循环程序实例

LOP: MOV AL, X[SI]

SHR BL, 1

JC SUB1

ADD AL, Y[SI]

JMP RES

SUB1: SUB AL, Y[SI]

RES: MOV Z[SI], AL

INC SI

LOOP LOP

MOV AH, 4CH

INT 21H

COSEG ENDS

END BEGIN

SI为数组下标

; 标志位送CF

; 为1, 转做减法

; 为0, 做加法

; 存结果

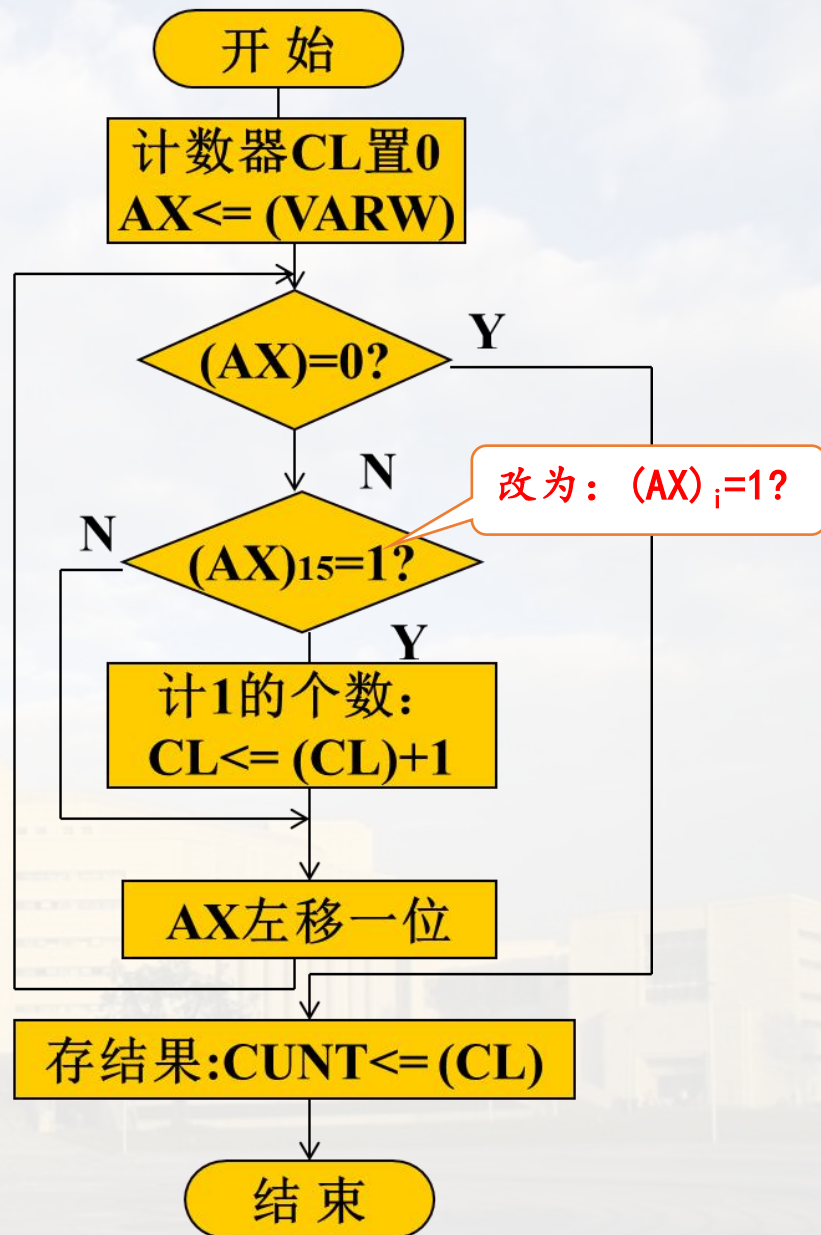
; 修改指针

4.13 循环程序实例

用上题方法试一试

例6 编写一程序，将字单元VARW内容用二进制表示时“1”的个数统计出来，存入CONT单元中。

通过将字单元逐位移入最高位来判断。为了减少循环次数，加上了判断各位是否全为0。当低位有多个全0时循环次数将减少。



4.13 循环程序实例



DATA SEGMENT

VARW DW 1101010010001000B

CONT DB ?

DATA ENDS

STACK1 SEGMENT PARA STACK

DW 20H DUP(0)

STACK1 ENDS

CODE SEGMENT

ASSUME CS:CODE, DS:DATA, SS:STACK1

BEGIN: MOV AX, DATA

MOV DS, AX

MOV CL, 0

MOV AX, VARW

;初始值为0, 统计1的个数

4.13 循环程序实例

```
LOP:TEST AX,0FFFFH ;测试 (AX) 是否为0, 可减少移位次数
      JZ  END0      ;为0, 循环结束
      JNS SHIFT     ;判最高位, 为0则转SHIFT
      INC CL        ;最高位为1, 计数
SHIFT:SHL AX,1
      JMP LOP
END0:MOV CONT,CL    ;存结果
      MOV AH,4CH
      INT 21H
CODE ENDS
      END BEGIN
```

SF: 0, 正, (AX) n=0, 不计数
1, 负, (AX) n=1, 计数

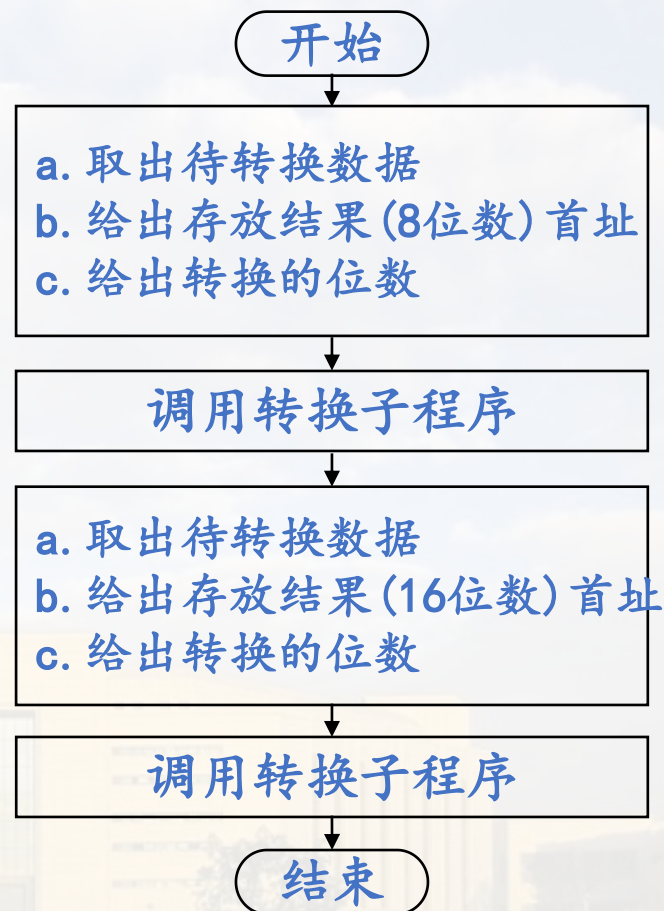
4.14

● 子程序设计实例

例：将两个给定的二进制数(8位和16位)转换为ASCII码字符串。

主程序提供：

- 1、被转换的数据；
- 2、转换后的ASCII码字符串的存储区的首地址



主程序框图

4.14 子程序设计实例

子程序完成二进制数与ASCII码字符串的转换。

子程序的入口参量有三个：

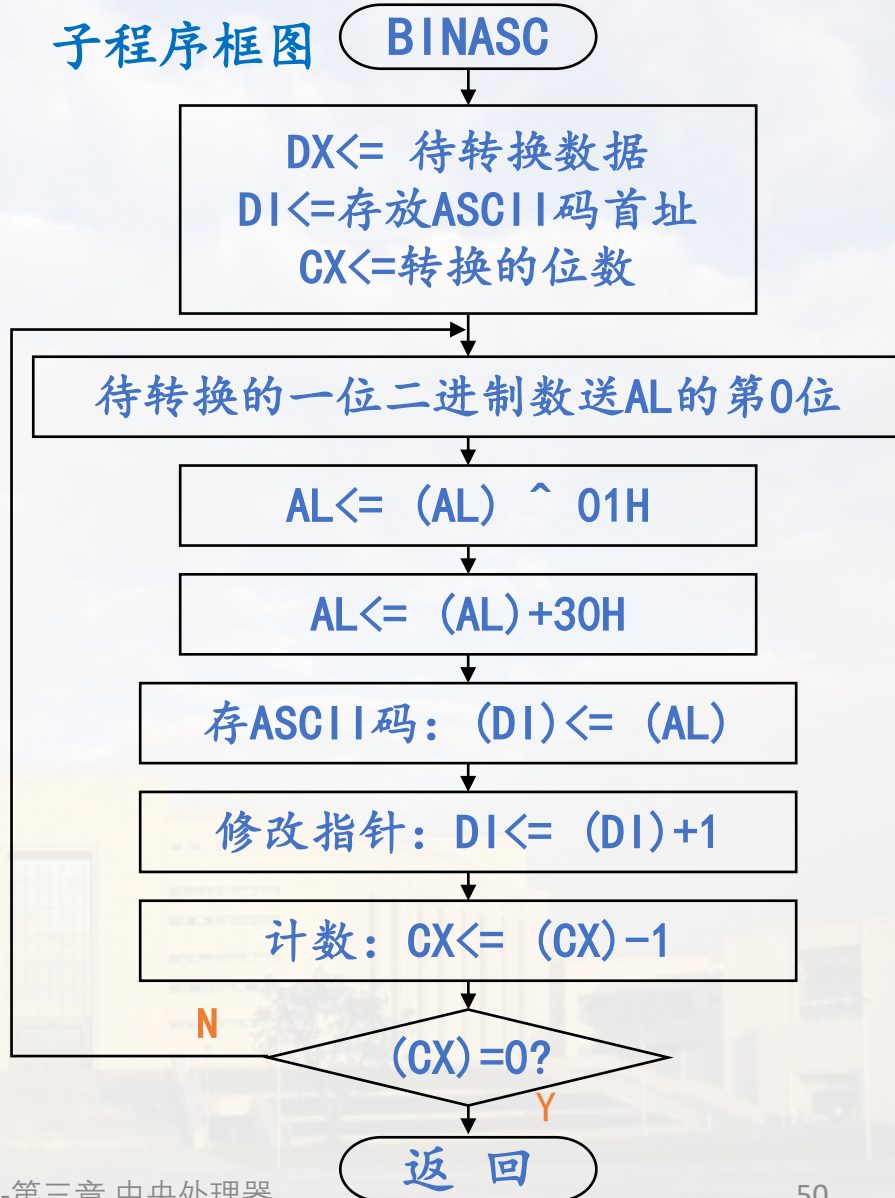
1. 被转换的数据；
2. 存储ASCII码字符串的首址；
3. 待转换的8/16位数。

无出口参量。

‘0’ 的ASCII码为30H,
‘1’ 的ASCII码为31H。

子程序框图

BINASC



源程序的数据段和堆栈安排如下：

```
DATA SEGMENT
```

```
    BIN1  DB 35H
```

```
    BIN2  DW 0AB48H
```

```
    ASCBUF DB 20H DUP(?)
```

```
DATA ENDS
```

```
STACK1 SEGMENT PARA STACK
```

```
    DW 20H DUP(0)
```

```
STACK1 ENDS
```

待转换的8位数，
待转换的16位数，

由于参量的传递方式有多种形式，其相应地在子程序中取入口参量的方法也有所不同。

三种参量传递方法：用寄存器传递参量，用堆栈传递参量，用地址表传递参量。

用法说明

三种参量传递方法的特点：

用**寄存器**传递参量：传递参数**较少**时使用，传递**参数本身**。

用**堆栈**传递参量：传递参数**较多**时使用，传递**参数本身**。

用**地址表**传递参量：传递参数**较多**时使用，传递**参数地址**。

4.14 子程序设计实例

1、用寄存器传递参量

主程序

设调用子程序时，入口参量为：被转换的数在DX中，若数位<16，则从高到低地存放，转换后的ASCII码的存放首址在DI中。信息的保存由主程序完成。

```
COSEG    SEGMENT
          ASSUME CS:COSEG, DS:DATA, SS:STACK1
START:   MOV  AX, DATA
          MOV  DS, AX
          XOR  DX, DX
          LEA  DI, ASCBUF    ;存放ASCII码的单元首址送DI
          MOV  DH, BIN1      ;待转换的第1个数据送DH
          MOV  AX, 8          ;待转换的二进制数的位数
          PUSH DI             ;保护信息
          CALL BINASC         ;调用转换子程序
          POP  DI             ;恢复信息
          MOV  DX, BIN2      ;待转换的第二个数据送DX
          MOV  AX, 16
          ADD  DI, 8          ;设置下一个数（16位数）的存放首址
          CALL BINASC
          MOV  AH, 4CH
          INT  21H
```

不送DL，减少子程序移位次数

转换子程序

```
BINASC PROC
    MOV CX, AX
LOP:ROL DX, 1                ; 8/16位数最高位移入最低位，首尾相连的小循环
    MOV AL, DL
    AND AL, 1                ; 保留最低位，屏蔽其它7位
    ADD AL, 30H              ; AL中即为该数字字符（0或1）的ASCII码
    MOV [DI], AL             ; 存结果
    INC DI                   ; 修改地址指针
    LOOP LOP
    RET
BINASC ENDP
COSEG ENDS
END START
```

此方法用R传递参数

主程序

1. BIN1/BIN2 (8/16位) $\longrightarrow$ DH/DX
; 待传送数据
2. 8/16位 $\longrightarrow$ AX; 待转换的位数
3. ASCBUF $\longrightarrow$ DI; 存放ASCII码的
单元首址送DI

子程序

1. 待转换数据在DX中
2. AX $\longrightarrow$ CX; 待转换的位数
3. DI; 存放ASCII码的单元首址
在DI中

2、用堆栈传递参量

如果使用堆栈传递参量，一般应包括：

- (1) 在主程序中，将待转换的数据、存放ASCII码的首址和转换的位数压入堆栈；
- (2) 在子程序中保存信息。

4.14 子程序设计实例

主程序

```
COSEG SEGMENT
ASSUME CS: COSEG, DS: DATA, SS: STACK1
BEGIN: MOV AX, DATA
```

8位数

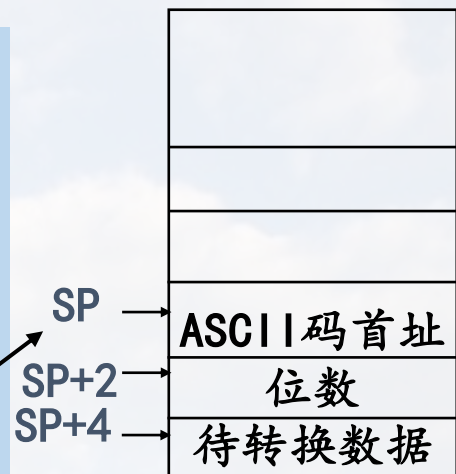
```
    MOV AH, BIN1
    PUSH AX          ; 待转换数据压栈
    MOV AX, 8
    PUSH AX          ; 转换位数压栈
    LEA DI, ASCBUF
    PUSH DI          ; 存放ASCII码的首址压栈
    CALL BINASC      ; 调用转换子程序
```

16位数

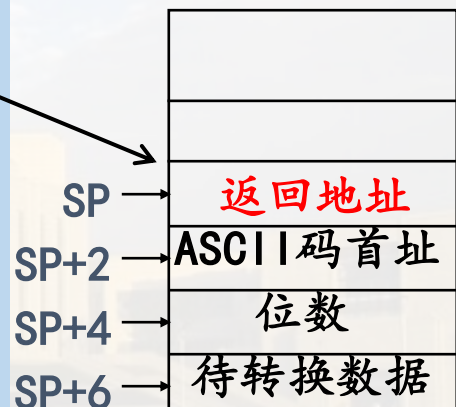
```
    MOV AX, BIN2
    PUSH AX
    MOV AX, 10H
    PUSH AX
    ADD DI, 8
    PUSH DI
    CALL BINASC
    MOV AH, 4CH
    INT 21H
```

调用子程序前还要保存断点, 段内调用

第二个数的地址要比第一个数的地址高8B



执行CALL指令
前堆栈情况



执行CALL指令
后堆栈情况

转换子程序

BINASC PROC

PUSH AX

PUSH CX

PUSH DX

PUSH DI

下面要使用这些R,
所以要保护

MOV BP, SP

MOV DI, [BP+10] ;从堆栈取入口参数

MOV CX, [BP+12]

MOV DX, [BP+14]

LOP: ROL DX, 1

MOV AL, DL

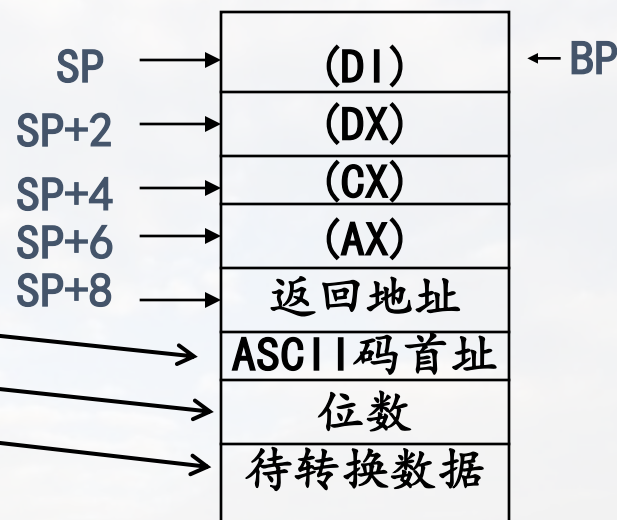
AND AL, 1

ADD AL, 30H

MOV [DI], AL

INC DI

LOOP LOP



子程序中保存信息并
执行MOV BP, SP后

4.14 子程序设计实例

POP DI

POP DX

POP CX

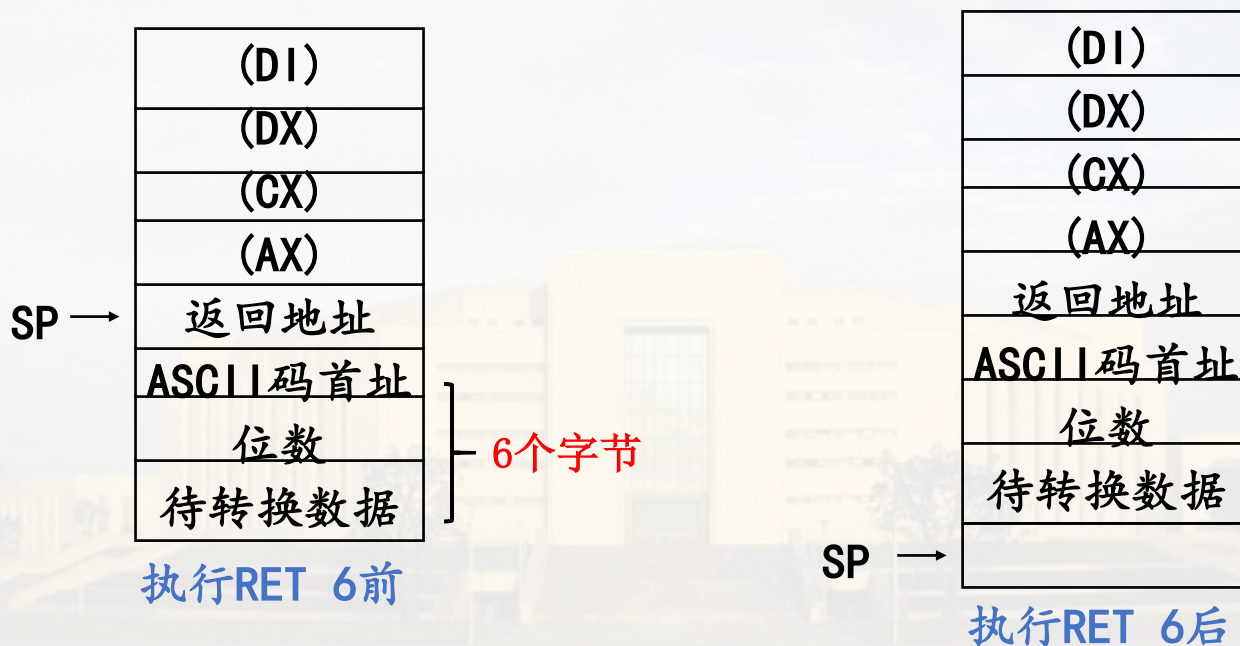
POP AX

RET 6 ; 返回并从堆栈中弹出6个字节

BINASC ENDP

COSEG ENDS

END BEGIN



3、用地址表传递参量

传递参数也可以采用传递参量的地址来实现。

在调用子程序前，将所有参量的地址依次存放在一个地址表中，将该表的首地址传送给子程序。

数据段部分改为：

DATA SEGMENT

BIN1 DB 35H

BIN2 DW 0AB48H

CUNT DB 8, 16

ASCBUP DB 20H DUP(?)

ADR_TAB DW 3 DUP(0) ; 存放参量地址表

DATA ENDS

主程序中有关指令序列修改为：

.....

MOV ADR_TAB, OFFSET BIN1 ; 存参量地址

MOV ADR_TAB+2, OFFSET CUNT

MOV ADR_TAB+4, OFFSET ASCBUP

MOV BX, OFFSET ADR_TAB ; 传表首址

CALL BINASC8

MOV ADR_TAB, OFFSET BIN2

MOV ADR_TAB+2, OFFSET CUNT+1

MOV ADR_TAB+4, OFFSET ASCBUP+8

MOV BX, OFFSET ADR_TAB ; 传表首址

CALL BINASC16

.....

4.14 子程序设计实例

转换子程序设置两个入口，一个是转换8位数据的入口BINASC8，另一个是转换16位数据的入口BINASC16。

```

BINASC  PROC
BINASC8: MOV DI, [BX] ; 取待转换8位数据
          MOV DH, [DI]
          JMP TRAN
BINASC16: MOV DI, [BX] ; 取待转换16位数据
          MOV DX, [DI]
TRAN:
          MOV DI, [BX+2] ; 取待转换数据位数
          MOV CL, [DI]
          XOR CH, CH
          MOV DI, [BX+4] ; 取存ASCII码首址
LOP:     ROL DX, 1
          MOV AL, DL ; 待转换的1位送到AL中转换
          AND AL, 1
          ADD AL, 30H ; 构成相应的ASCII码
          MOV [DI], AL ; 存结果
          INC DI
          LOOP LOP
          RET
BINASC  ENDP
    
```

第4章作业:

4. 2, 4. 6, 4. 7, 4. 8, 4. 11, 4. 16, 4. 1

7, 4. 18, 4. 20



THANK

@刘辉

